LPP PREGUNTAS TEMA #04

1. **¿Qué es una clase?**

Una clase es tipo de estructura que define las propiedades (atributos) y comportamientos (métodos) comunes de un conjunto de objetos.

2. **¿Qué es un objeto?**

Un objeto es una instancia concreta de una clase, esta contiene datos específicos y puede ejecutar los métodos definidos en la clase.

3. **¿Qué es una variable local?**

Es una variable que se define dentro de un método y solo es accesible dentro de ese método.

4. **¿Qué es una variable de instancia?**

Es una variable que pertenece a una instancia de una clase y es accesible dentro de los métodos de esa instancia.

5. **¿Qué es una variable global?**

Es una variable que puede ser accedida desde cualquier parte del programa.

6. **¿Qué es una variable de clase?**

Es una variable que pertenece a la clase y es compartida por todas sus instancias.

7. **¿Qué es un método de acceso (getter)?**

Es un método que permite acceder al valor de una variable de instancia desde fuera de la clase.

8. **¿Qué es un método de asignación de valores (setter)?**

Es un método que permite modificar el valor de una variable de instancia desde fuera de la clase.

9. **¿Cuál es la visibilidad del método “initialize”?**

Es un método privado, solo puede ser llamado automáticamente al crear un objeto con “.new”.

10. **El valor retornado por “initialize” es usado para la construcción del objeto. ¿Verdadero o falso?**

Falso. Aunque `initialize` se ejecuta durante la construcción del objeto, su valor retornado no se utiliza.

11. **Considere el siguiente código Ruby:**

```
1  class AClass
2        @x = 4; @y = 9
3
4        def initialize(x,y)
5              @x, @y = x, y
6        end
7  end
```

   a. **¿Qué tipo de variable son las de la línea 2? ¿Cuál es su visibilidad?**

Son variables de instancia de la clase, su visibilidad está limitada al contexto de la clase, en este caso no son accesibles.

   b. **¿Qué tipo de variable son las de la línea 5? ¿Cuál es su visibilidad?**

Son variables de instancia, accesibles dentro de los métodos de instancia de esa clase.

12. **Señale el error de concepto en la codificación de la siguiente clase Ruby:**

```
class MyClass
        def initialize(b)
              @b = b
        end
        def to_s
              puts @b
        end
end
```

El método “to_s” debe devolver un valor tipo cadena, pero está usando “puts”, que imprime en consola y devuelve “nil”. Se debe usar “return @b” o simplemente “@b” (al ser la última linea de codigo el return es implícito).

13. **Considere el siguiente código Ruby:**

```
1  class AClass
2        attr_accessor :n
3 ...
4        def initialize(n)
5              @n = n
6        end
7 ...
8        def square
9              n = n * n
10       end
11 ...
12  end
```

En los accesos a n a la derecha de la igualdad en la línea 9:

   a. **¿A qué tipo de variable se está accediendo?**

A una variable local “n” (Ruby interpreta “n =” como el setter).

b. ¿Y en el acceso a la izquierda de la igualdad?

También a una variable local “n”, no al atributo “@n”.

c. ¿”attr_accessor” es un método de instancia o de clase?

Es un método de clase.

d. ¿En qué clase está definido “attr_accessor”?

En la clase “Module”.

14. ¿En qué forma se define en Ruby un método de clase?

Usando “def self.metodo” o “def NombreClase.metodo”.

15. ¿Es posible definir una constante de la clase “MyClass” antes de la definición del método “initialize”?

Sí, las constantes pueden definirse en cualquier parte del cuerpo de la clase.

16. ¿A qué clase de variable “n· permite acceder la declaración de la línea 3?

```
1  class Tutu
2        class << self
3                attr_accessor :n
4        end
5  end
```

A una variable de clase (usada como variable de clase singleton del objeto “Tutu”, es decir, “Tutu.n” y “Tutu.n=”).

17. ¿Qué se entiende por herencia?

Es un mecanismo por el cual una clase puede heredar atributos y métodos de otra clase.

18. Enumere los tipos de herencia y descríbalos brevemente.

- **Simple:** una clase hereda de una sola superclase.
- **Múltiple:** una clase hereda de varias clases (no soportada directamente en Ruby).
- **Multinivel:** una clase hereda de otra, que a su vez hereda de otra.
- **Jerárquica:** varias clases heredan de una misma superclase.

19. ¿Qué tipo de herencia proporciona Ruby?

Herencia simple, pero permite compartir comportamiento mediante mix-ins (módulos).

20. ¿Qué se entiende por invalidación de un método (overriden)?

Reescribir un método en una subclase que ya estaba definido en una superclase.

21. ¿Qué puede ocurrir si en una subclase se escribe un método con nombre “intimo” igual al de un método privado “intimo” de la superclase? ¿Qué ocurre si el método “toto” de la superclase llama a “intimo”?

La subclase no podrá sobrescribir correctamente el método “intimo” si era privado, y si “toto” llama a “intimo”, se ejecutará el método privado original, no el sobrescrito.

22. ¿Qué ocurre cuando se llama a "super" sin argumentos?

Se pasan automáticamente los mismos argumentos recibidos por el método actual al método de la superclase.

23. ¿Cómo se puede llamar a "super" sin argumentos?

Escribiendo "super" sin paréntesis ni argumentos.

24. Describa la salida de los dos programas Ruby:

```ruby
class A
  @@value = 1
  def A.value
    @@value
  end
end

puts A.value

class B < A
  @@value = 2
end

puts A.value
puts B.value

class C < A; @@value = 3; end

puts B.value
puts C.value

puts A.class_variables.inspect
puts B.class_variables.inspect
puts C.class_variables.inspect
```

```ruby
class A
  @value = 1
  def self.value
    @value
  end
end

puts A.value

class B < A
  @value = 2
end

puts A.value
puts B.value

class C < A; @value = 3; end

puts B.value
puts C.value
```

- En el primero, al modificar "@@value" en B o C, cambia para todas las clases ya que es compartida. Por tanto los "puts" mostrarían: "1", "22" y "333".
- En el segundo, "@value" es una variable de clase singleton y se mantiene independiente para cada clase. Por tanto los "puts" mostrarían: "1", "12" y "123".

25. Suponga que la clase B hereda de A un método "tutu" que usa la constante "C" definida en A. Si en la clase B se define "C", ¿Qué definición de "C" usará "tutu", la de A o la de B?

Usará la constante "C" de B, ya que Ruby busca constantes en el contexto de la subclase primero.

26. ¿Qué es la encapsulación?

Es el principio de ocultar los detalles internos de un objeto, exponiendo solo lo necesario mediante métodos públicos.

27. ¿Cuál es la visibilidad por defecto de un método?

Pública, salvo “initialize” que es privado por defecto.

ALGORITMO DE BÚSQUEDA

1º Eigenclass.
2º Métodos de instancia de la clase.
3º Módulos de la clase.
4º Métodos de instancia de la superclase (clase del padre).
5º Módulos de la superclase (clase padre).
6º Buscar en las superclases: Object y BasicObject.
7º Si no lo encuentra, llama al método “method_missing”.

---

74. Describa el algoritmo de búsqueda de Ruby para el método “my_method()”:

```
class MyClass
        def my_method; 'my_method()'; end
end
class MysubClass < MyClass
end
obj = MySubclass.new
obj.my_method()
```

1º Busca el método “my_method()” en la **eigenclass del objeto** “obj”, no lo encuenta.
2º Busca el método “my_method()” en los **métodos de instancia de la clase** “MySubClass”, no lo encuenta.
3º Busca el método “my_method()” en los **módulos incluidos en la clase** “MySubClass”, no hay ningún módulo incluído, así que no encuentra el método.
4º Busca el método “my_method()” en los **métodos de instancia de la superclase** “MyClass”, y aquí es donde lo encuentra.

75. Describa el algoritmo de búsqueda de Ruby para el método "my_method()":

```
module M
        def my_method; 'M#my_method()'; end
end
class C
        include M
end
class D < C ; end
D.new.my_method()
```

1º Busca el método "my_method()" en **eigenclass del objeto** "D.new", no lo encuentra.
2º Busca el método "my_method()" en los **métodos de instancia de la clase** "D", no lo encuentra.
3º Busca el método "my_method()" en los **módulos incluidos en la clase** "D", no hay módulos incluidos, así que no lo encuentra.
4º Busca el método "my_method()" en los **métodos de la instancia de la superclase** "C", no lo encuentra.
5º Busca el método "my_method()" en los **módulos incluídos en la superclase** "C", ahí encuentra el método.

76. Considere el siguiente código Ruby:

```
1 def Integer.a_method(x)
2       x.to_i
3 end
4 n = Fixnum.a_method("5")
```

Describa el algoritmo de búsqueda de la invocación de la línea 4.

1º Busca el método "a_method()" en **eigenclass** del objeto "Fixnum", no lo encuentra.
2º Busca el método "a_method()" en los **métodos de la instancia de la clase** "Fixnum", no lo encuentra.
3º Busca el método "a_method()" en los **módulos incluidos en la clase** "Fixnum", no lo encuentra.
4º Busca el método "a_method()" en los **métodos de la instancia de la superclase** "Integer", no lo encuentra.
5º Busca el método "a_method()" en los **módulos incluidos en la superclase** "Integer", es ahí donde lo encuentra.